

Projekt zaliczeniowy



Programowanie obiektowe
Rok akademicki 2024/2025

Autorzy:

1. Emilia Lehmann
2. Natalia Maj
3. Weronika Kościug

System biletów lotniczych

W dzisiejszych czasach bardzo często podróżujemy samolotami. Nasz projekt ma za zadanie obsłużyć system kupowania biletów. Można w nim uzupełnić dane kupującego, szczegóły podróży oraz dodać dodatkowe usługi. System przelicza cenę, sprawdza dostępność miejsc w samolocie i rezerwuje miejsce.

Podział ról

Emilia Lehmann: Klasy: Bilet, BiletKrajowy, BiletMiędzynarodowy, BiletMiędzynarodowyKontynentalny, BiletZPrzesiadkami, BlednaDataLotuException, BrakMiejscException, IUslugowy, + instrukcja + część dokumentacji XML

Natalia Maj: Cała praca nad interfejsem graficznym + dodanie elementów dodatkowych (m.in. GIF) + Testy jednostkowe podstawowych klas systemu + część dokumentacji XML

Weronika Kościug: Klasy: BiletDbContext, BiletPoDacieComparer, BledneMiastoException + odczyt danych do/z pliku (serializacja XML) + Wykorzystanie interfejsu ICloneable i IComparer + Diagram UML + wykorzystanie jednego delegatu + część dokumentacji XML

Opis i rola klas

System składa się z 12 klas. Klasa abstrakcyjna Bilet, 4 klasy dziedziczące po klasie Bilet, 3 klasy wyjątków, interfejs IUslugowy, BiletPoDacieComparer, BiletDbContext i Program.

Klasa bilet:

Opis: Klasa bilet jest klasą abstrakcyjną, jest fundamentem całego projektu systemu rezerwacji lotów. Służy jako szablon dla konkretnych typów biletów (krajowych, międzynarodowych itp.), definiując wspólny zestaw cech.

Rola: Wymusza na klasach dziedziczących po niej posiadanie kluczowych danych (standaryzacja). Klasa centralizuje mechanizmy takie jak generowanie unikalnego numeru lotu, walidację dat, zarządzanie pulą dostępnych miejsc oraz obsługę trwałości danych (XML i Baza Danych). Przez strukturę *List<Bilet>* system może zarządzać różnymi rodzajami biletów w jednej kolekcji, wywołując na nich te same metody (np. obliczanie ceny końcowej), które zachowują się różnie w zależności od typu obiektu (polimorfizm).

Modyfikatory dostępu do pól: private- Większość pól (np. cena, *imiePasazera*) jest prywatna, chroni to dane przed niekontrolowaną modyfikacją z zewnątrz.

static *licznikiMiejsc*- pole współdzielone przez wszystkie instancje, co pozwala śledzić zajętość miejsc na konkretny lot niezależnie od tego, który obiekt biletu jest tworzony.

Właściwości: Zapewniają kontrolowany dostęp do pól prywatnych. Na przykład w *DataWylotu*, *MiastoWylotu* i *MiastoPrzylotu* dodano walidację (throw new Exception), co zapobiega wprowadzeniu do systemu błędnych danych (np. daty z przeszłości). We właściwości *numerLotu* ustawiamy formułę tworzącą unikalny numer dla każdego lotu (wykorzystującą nazwę miasta przylotu, miasta wylotu, datę i godzinę wylotu).

Dodajemy właściwość string *GodzinaWylotu* na potrzeby zapisu do pliku Xml, gdyż format ten nie jest dobrze odczytywany. Serializator ignoruje pierwszą właściwość TimeOnly *GodzinaWylotu* i używa drugiej (zasługa dodania [XmlElement("GodzinaWylotu")] którą odpowiednio przygotowaliśmy w formacie string, a później ponownie zmieniamy.

Interfejsy, które implementuje: *IComparable* umożliwia korzystanie z metody *CompareTo*, *IEquatable* umożliwia korzystanie z metody *Equals*, *ICloneable* umożliwia korzystanie z metody *Clone*.

Metody Wirtualne:

- *MnoznikSezonowy* ustawia mnożnik na podstawie czasu, w którym chcemy zarezerwować lot -w okresie wakacji cena biletu jest 1,5 razy większa niż w innym czasie, w okolicach świąt cena też jest wyższa, bilet może być droższy od 1,3 do 1,8 razy droższy.
- *MnoznikKlasy*- klasa ekonomiczna ma mnożnik 1, natomiast klasa biznesowa jest 1,5 razy droższa, a pierwsza klasa 2 razy droższa.
- *ObliczCeneKoncowa*- zwraca cenę początkową przemnożoną przez wymienione wcześniej mnożniki.

- *ToString* tworzy tekst z danymi pasażera i danymi lotu, które będą się wyświetlać po uruchomieniu programu.

Metody:

- *CompareTo* porównuje bilety na podstawie ceny.
- *Equals* sprawdza czy jeden bilet jest równy innemu.
- *Clone* klonuje bilety (płytką kopia).
- *CloneZNowaGodnoscia* klonuje bilet, ale dla innej osoby (takie same dane lotu, ale dla kogoś innego), przy czym sprawdza czy nie zostanie przekroczona liczba miejsc w samolocie.
- *doListyBiletow* dodaje bilet do listy kupionych biletów.
- *wyswietlKupioneBilety* wyświetla listę wszystkich kupionych biletów.
- *ZapisXML* zapisuje listę biletów w pliku XML.
- *OdczytajXML* odczytuje listę biletów z pliku XML.
- *AktualizacjaZOdczytu* dodajemy odczytane bilety z pliku XML do listy zakupionych biletów, sprawdzamy czy na któryś lot nie zostanie przekroczona liczba miejsc.
- *SaveToDB* zapisuje dane do bazy danych.
- *ZastosujRabat* – metoda pobiera delegat jako parametr i stosuje metodę dla właściwości *Cena*

Delegat:

DelegatZnizka – przyjmuje wszystkie metody, które przyjmują *double* i zwracają również *double*

Klasa Znizki:

Opis: Klasa pomocnicza, statyczna, aby móc łatwo skorzystać z metod tam zawartych bez tworzenia instancji klasy. Jej główny cel to odciążyć klasę bilet, dlatego to tam będziemy zawierać metody obliczające zniżkę z pierwotnej ceny. Łatwo ją rozbudować o inne metody ze zniżkami.

Metody:

- *Student* - przemnaża liczbę przez 0.5
- *Senior* - przemnaża liczbę przez 0.7

Klasa BiletKrajowy:

Opis: Klasa ta dziedziczy po klasie *Bilet*. Reprezentuje ona rezerwację na loty odbywające się wewnątrz granic jednego państwa. Rozszerza ona funkcjonalność klasy bazowej o specyficzne reguły podatkowe dotyczące lotów lokalnych.

Rola: Głównym powodem stworzenia tej klasy jest konieczność uwzględnienia krajowej stawki podatku VAT (8%). Klasa pozwala na hermetyzację danych dotyczących podatków krajowych, które nie powinny obciążać struktury ogólnej dla wszystkich biletów. Dzięki nadpisaniu metod wirtualnych, system może traktować ten bilet jak każdy inny, ale podczas obliczania sumy w koszyku, automatycznie doliczy odpowiedni podatek bez ingerencji użytkownika (implementacja polimorfizmu).

Modyfikatory dostępu do pól: `private double stawkaPodatkowa`- Zastosowano modyfikator prywatny, aby uniemożliwić przypadkową zmianę wysokości podatku z zewnątrz. Domyślna wartość 0.08 (8%) jest zdefiniowana bezpośrednio w klasie.

Nadpisywanie metod: Metoda najpierw wywołuje `base.ObliczCeneKoncowa()`, co pozwala uwzględnić mnożniki sezonowe i klasowe zdefiniowane w klasie Bilet, a następnie mnoży ten wynik przez stawkę podatku krajowego.

Wykorzystuje metodę `base.ToString()`, do której dołącza informację o stawce podatkowej sformatowaną jako wartość procentową.

Klasa BiletMiedzykrajowy:

Opis: Jest to klasa, dziedzicząca po klasie bazowej Bilet. Reprezentuje ona dokument podróży, jakim jest bilet na lot pomiędzy różnymi krajami (w obrębie tego samego kontynentu).

Rola: Główną rolą tej klasy jest obsługa specyfiki lotów międzynarodowych, która wiąże się z dodatkowymi kosztami (np. opłaty lotniskowe, podatki transgraniczne). Dzięki wydzieleniu jej jako osobnej klasy, system może automatycznie różnicować sposób naliczania opłat bez modyfikowania głównej logiki klasy Bilet.

Modyfikatory dostępu do pól: `private double dodatkoweOpłaty` - Zastosowano modyfikator prywatny, aby uniemożliwić zmianę z zewnątrz. Domyślna wartość 50 jest zdefiniowana bezpośrednio w klasie.

Nadpisywanie metod: Metoda najpierw wywołuje `base.ObliczCeneKoncowa()`, co pozwala uwzględnić mnożniki sezonowe i klasowe zdefiniowane w klasie Bilet, a następnie dolicza do niej specyficzną opłatę międzynarodową. Wykorzystuje metodę `base.ToString()`, do której dołącza informację o dodatkowych kosztach.

Klasa BiletMiedzykontynentalny:

Opis: Jest to klasa, dziedzicząca po klasie bazowej Bilet. Rozszerza klasę bazową o specyfikę lotów dalekobieżnych, czyli zawiera informacje o konieczności posiadania wizy oraz pozwala zamówić posiłki na dany lot.

Rola: Główną rolą tej klasy jest obsługa specyfiki lotów międzykontynentalnych. Zawiera informację o konieczności posiadania wizy, do państwa, gdzie lecimy oraz informacje o ewentualnym zamówieniu przez pasażera posiłków. Wyświetla też informację o cenie usług dodatkowych (posiłków).

Modyfikatory dostępu do pól: Pola `wizaWymagana` oraz `cena UslugDodatkowych` są prywatne, chronimy w ten sposób integralność danych – cena usług dodatkowych może być modyfikowana tylko przez dedykowane metody klasy.

Nadpisywanie metod: Metoda najpierw wywołuje `base.ObliczCeneKoncowa()`, co pozwala uwzględnić mnożniki sezonowe i klasowe zdefiniowane w klasie Bilet, a następnie dolicza do niej 200 zł, jeśli wymagana jest wiza oraz dodaje cenę usług dodatkowych. Wykorzystuje metodę `base.ToString()`, do której dołącza informację o tym czy wiza jest wymagana oraz o cenie usług dodatkowych.

Metody:

- *DodajPosilek* dodaje posiłek do listy oraz zwiększa *CenaUslugDodatkowych* o odpowiednią kwotę.
- *WyswietlUslugi* wypisuje posiłki, które zamówił pasażer.

Klasa BiletZPrzesiadkami:

Opis: Jest to klasa, dziedzicząca po klasie bazowej *Bilet*, która zamiast pojedynczej trasy, zarządza kolekcją. Reprezentuje złożony produkt lotniczy, który składa się z co najmniej dwóch oddzielnych lotów (etapów) realizowanych w ramach jednej rezerwacji.

Rola: Głównym celem klasy jest umożliwienie tworzenia tras, których nie da się obsłużyć pojedynczym połączeniem (np. lot z Krakowa do Nowego Jorku z przesiadką w Frankfurtach). Klasa pełni rolę kontenera, który sumuje koszty poszczególnych składowych (biletów krajowych, międzynarodowych itp.) i stosuje wobec nich zniżkę.

Modyfikatory dostępu do pól: *private double zniżka* -stała (domyślnie 10%), która promuje kupowanie lotów łączonych. Użytkownik nie może jej zmienić bez użycia dedykowanej właściwości.

Nadpisywanie metod: *ObliczCeneKoncowa* oblicza końcową cenę biletu z przesiadkami jako sumę cen poszczególnych etapów z uwzględnieniem zniżki.

Metoda: *GenerujNumerTrasy* generuje numer trasy na podstawie numerów lotów poszczególnych etapów podróży.

Klasa BlednaDataLotuException:

Opis: Klasa wyjątku, która dziedziczy po podstawowej klasie *Exception*. Została stworzona, aby obsługiwać konkretny scenariusz błędu w systemie: próbę ustawienia daty lotu, która jest nieprawidłowa (np. data z przeszłości).

Rola: W klasie *Bilet*, w właściwości *DataWylotu*, system sprawdza czy data, którą chcemy wpisać jest w przeszłości. Jeśli tak, "rzuca" ten konkretny wyjątek. Dzięki temu system blokuje możliwość zakupu biletu na lot, który już się odbył. Zamiast używać ogólnej klasy *Exception*, użycie tej klasy pozwala na precyzyjne przechwytywanie (catch) tylko błędów związanych z czasem lotu, bez wpływania na inne błędy systemowe.

Klasa BledneMiastoException:

Opis: Klasa wyjątku, która dziedziczy po podstawowej klasie *Exception*. Została zaprojektowana do obsługi błędów walidacji danych geograficznych w systemie: próbę ustawienia nazwy miasta (przylotu lub wylotu), która jest krótsza niż 3 znaki.

Rola: System wymaga, aby nazwy miast miały co najmniej 3 znaki (wymagane do poprawnego wygenerowania *NumeruLotu*). Wyjątek ten zapobiega błędom systemowym, które wystąpiłyby przy próbie przetworzenia zbyt krótkich nazw. Jest rzucany bezpośrednio wewnątrz właściwości *MiastoWylotu* i *MiastoPrzylotu*. Zamiast używać ogólnej klasy *Exception*, użycie tej klasy pozwala na precyzyjne przechwytywanie (catch) tylko błędów związanych z czasem lotu, bez wpływania na inne błędy systemowe.

Klasa BrakMiejscException:

Opis: Klasa wyjątku, która dziedziczy po podstawowej klasie Exception. Służy do sygnalizowania sytuacji krytycznej: próby dokonania rezerwacji na lot, na który wyprzedano już wszystkie dostępne miejsca.

Rola: Jest to kluczowy element mechanizmu kontroli zajętości samolotu. Wyjątek ten jest rzucany w konstruktorze klasy Bilet oraz w metodzie *CloneZNowaGodnoscia*. Przerwanie działania konstruktora tym wyjątkiem gwarantuje, że niepoprawny bilet nie zostanie dodany do listy *kupioneBilety*. Zamiast używać ogólnej klasy Exception, użycie tej klasy pozwala na precyzyjne przechwytywanie (catch) tylko błędów związanych z ilością miejsc w samolocie, bez wpływania na inne błędy systemowe.

Klasa IUslugowy:

Opis: Interfejs, który definiuje zbiór wymagań dla klas biletów, które oferują pasażerom dodatkowe, płatne świadczenia wykraczające poza sam przelot. Określa on zestaw metod i właściwości niezbędnych do obsługi cateringu oraz rozliczania kosztów dodatkowych.

Rola: Interfejs pozwala na oddzielenie logiki transportowej od logiki usługowej. Nie każdy bilet musi oferować posiłki (np. linie krajowe), ale te, które to robią (np. *BiletMiedzykontynentalny*), muszą implementować ten interfejs.

Klasa BiletPoDacieComparer:

Opis: Klasa pomocnicza implementuje interfejs IComparer<Bilet> i służy do porównywania dwóch obiektów klas na podstawie daty wylotu.

Metoda: Użycie metody Compare pozwala nam posortować wszystkie nasze bilety od najwcześniejszych do najpóźniejszych.

Klasa BiletDbContext:

Opis: Klasa dziedziczy po DbContext, dzięki temu zarządza bazą danych przy użyciu EntityFramework. Centralny punkt odpowiedzialny za mapowanie obiektów klas biletów na wiersze w tabeli bazy danych.

Opis interfejsu użytkownika (GUI) i testów jednostkowych

GUI

Aplikacja składa się z trzech głównych zakładek:

1. **Nowy bilet** – dodawanie nowych biletów z możliwością wyboru typu biletu, klasy, daty i godzin lotu oraz dodatkowych usług.
2. **Lista biletów** – przeglądanie listy wszystkich zakupionych biletów i podgląd ich szczegółów.

3. **Zapis / Odczyt** – zapis listy biletów do pliku XML oraz odczyt z pliku XML.

Aplikacja obsługuje różne typy biletów: krajowe, międzykrajowe i międzykontynentalne. Bilety międzykontynentalne mogą mieć dodatkowo wymaganie wizy i możliwość wyboru posiłków.

Interfejs użytkownika (MainWindow.xaml)

Zakładka „Nowy bilet”

Dane pasażera – imię i nazwisko (TextBox).

Dane lotu – miasto wylotu i przylotu, data wylotu (DatePicker), godzina (TextBox).

Typ biletu – wybór typu biletu (ComboBox) i klasy (ComboBox).

Dodatkowe opcje – wiza i posiłki (tylko dla biletów międzykontynentalnych).

Podgląd ceny – dynamiczne wyświetlanie ceny w zależności od wyborów.

Dodawanie biletu – przycisk „Dodaj bilet” zapisuje dane i aktualizuje listę.

Zakładka „Lista biletów”

ListView – wyświetla wszystkie bilety z informacjami: numer biletu, pasażer, miasto wylotu/przylotu, data, cena.

Przycisk „Pokaż szczegóły” – wyświetla pełne informacje o zaznaczonym bilecie.

Zakładka „Zapis / Odczyt”

Zapis do XML – zapisuje listę biletów do wskazanego pliku.

Odczyt z XML – wczytuje listę biletów z pliku i wyświetla w ListView.

Nazwa pliku – pole tekstowe z nazwą pliku, domyślnie bilety.xml.

Logika aplikacji (MainWindow.xaml.cs)

Inicjalizacja

- Wypełnia ComboBoxy dla typu biletu i klasy.
- Ładuje listę istniejących biletów.

Obsługa wyboru typu biletu

- Bilety międzykontynentalne pokazują dodatkowe opcje: wizę i posiłki.
- Zmiana typu biletu aktualizuje widoczność odpowiednich kontroltek.

Dodawanie biletu

- Pobiera dane z pól tekstowych, ComboBoxów i DatePicker.
- Tworzy obiekt odpowiedniego typu biletu (BiletKrajowy, BiletMiedzykrajowy, BiletMiedzykontynentalny).
- Dla biletów międzykontynentalnych dodaje posiłki, jeśli zaznaczone.
- Po dodaniu biletu lista jest odświeżana, a użytkownik otrzymuje komunikat o sukcesie.

Obliczanie ceny (Podgląd ceny)

- Cena bazowa: 300 PLN.
- Uwzględnia sezon (lipiec–sierpień: +50%).

- Uwzględnia klasę: ekonomiczna, biznesowa, pierwsza.
- Bilety międzykontynentalne: wiza +200 PLN, posiłki +50 PLN każdy.
- Cena jest aktualizowana na bieżąco po zmianach w polach.

Zapis i odczyt z XML

- Funkcje Bilet.ZapisXML i Bilet.OdczytajXML zapisują i wczytują listę biletów.
- ListView jest odświeżany po każdym zapisie/odczycie.

Testy jednostkowe

Testy są napisane przy użyciu frameworka **MSTest** i sprawdzają poprawność klas biletów, wyjątków i interfejsu usług dodatkowych.

Bilety krajowe (BiletKrajowyTests)

- Sprawdzenie stawki podatkowej (domyślna i zmiana).
- Obliczanie ceny końcowej z uwzględnieniem podatku, klasy i sezonu.
- Metoda ToString zawiera informacje o podatku.

Bilety międzykontynentalne (BiletMiedzykontynentalnyTests)

- Wiza i cena usług dodatkowych domyślnie.
- Dodawanie posiłków zwiększa cenę usług dodatkowych.
- ToString zawiera informacje o wymaganej wizie i wybranych posiłkach.

Bilety międzykrajowe (BiletMiedzykrajowyTests)

- Sprawdzenie dodatkowych opłat i ich uwzględnianie w cenie końcowej.
- Możliwość zmiany dodatkowych opłat.
- ToString pokazuje informacje o dodatkowych opłatach.

Porównywanie biletów (BiletPoDacieComparerTests)

- Porównuje bilety po dacie wylotu.
- Sprawdza zachowanie dla null i takich samych dat.

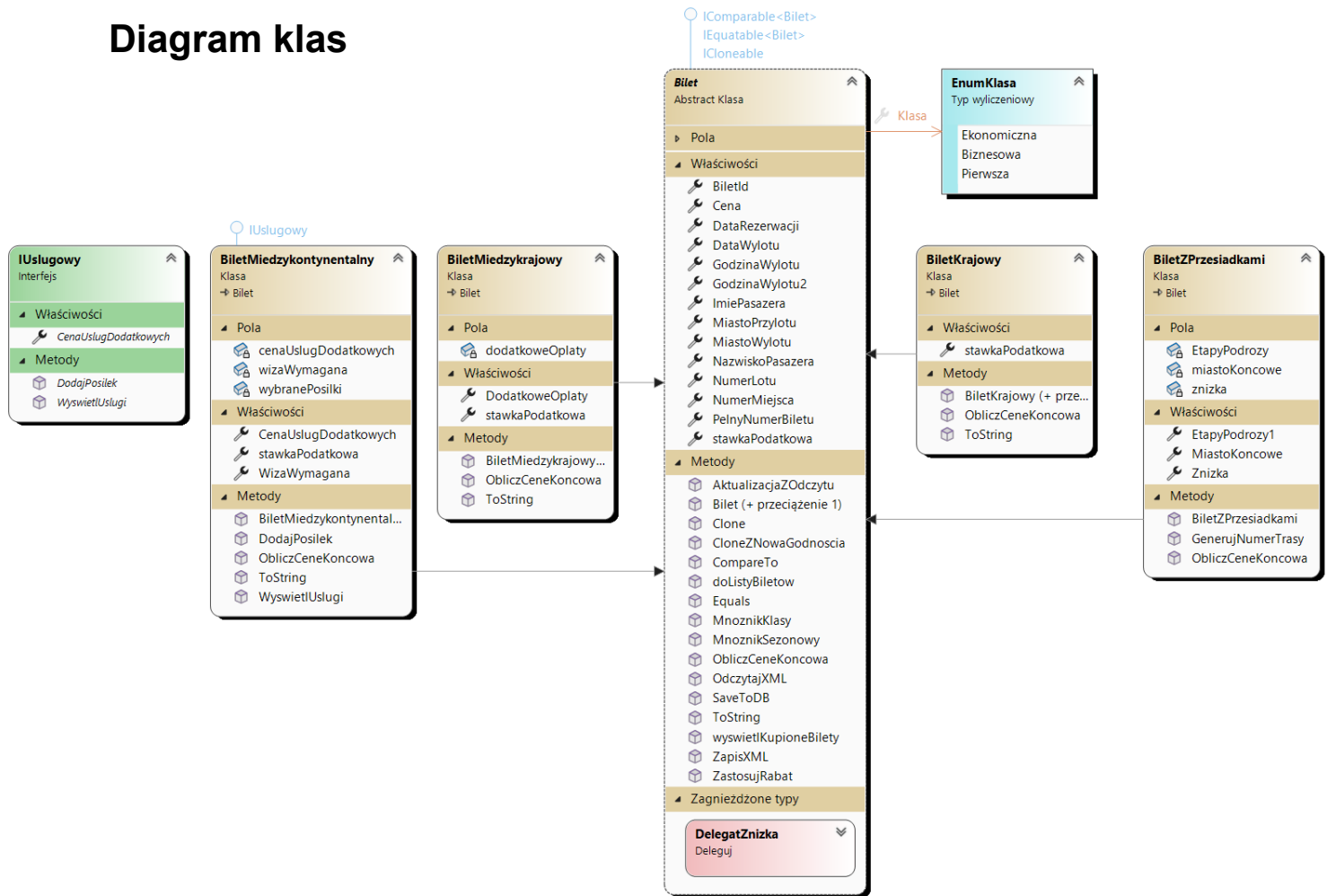
Obsługa wyjątków

- **BlednaDataLotuException** – data wylotu w przeszłości.
- **BledneMiastoException** – zbyt krótkie nazwy miast.
- **BrakMiejscException** – przekroczenie maksymalnej liczby miejsc (180).

Interfejs usług dodatkowych (IUslugowyTests)

- Sprawdza dodawanie posiłków i zwiększanie ceny usług dodatkowych

Diagram klas



Opis funkcjonalności

1. Zarządzanie rodzajami biletów (polimorfizm)

System obsługuje różne typy podróży, z których każdy posiada własną logikę obliczania ceny i specyficzne cechy.

Bilety Krajowe: Obsługa lotów lokalnych z automatycznym naliczaniem podatku VAT (8%).

Bilety Międzykontynentalne: Obsługa lotów długodystansowych z weryfikacją wymogu wizowego i zarządzaniem cateringiem.

Bilety z Przesiadkami: Agregacja wielu lotów w jedną trasę z naliczaniem rabatu za połączenie.

- **Klasy:** Bilet (bazowa), BiletKrajowy, BiletMiedzykrajowy, BiletMiedzykontynentalny, BiletZPrzesiadkami.

2. Zarządzanie procesem rezerwacji i miejscami

System automatyzuje proces przypisywania pasażerów do konkretnych lotów, dbając o unikalność rezerwacji i nieprzekraczanie limitów miejsc.

Generowanie unikalnego numeru lotu: Automatyczne tworzenie identyfikatora na podstawie miast, daty i godziny (np. KRA-WAR-20251220-1530).

Automatyczne przydzielanie miejsc: Dzięki statycznemu słownikowi licznikMiejsc system śledzi zajętość dla każdego lotu z osobna i nadaje kolejny numer miejsca.

Walidacja dostępności: Mechanizm blokuje sprzedaż biletów po osiągnięciu limitu (MAX_MIEJSC).

- **Klasy:** Bilet, BrakMiejscException.

3. Dynamiczne obliczanie cen i system rabatowy

System realizuje złożoną politykę cenową, która dostosowuje koszt biletu do czynników rynkowych i rodzaju podróży.

Mnożniki sezonowe: Automatyczne podwyżki w okresach szczytowych (wakacje, Święta).

Mnożniki klas: Różnicowanie cen dla klas Ekonomicznej, Biznesowej i Pierwszej.

Podatki i Zniżki: Obsługa specyficznych stawek podatkowych dla lotów krajowych oraz zniżek za loty łączone.

Elastyczne Rabaty: Możliwość nakładania zniżek (np. Student, Senior) poprzez mechanizm delegatów.

- **Klasy:** Bilet, BiletKrajowy, BiletZPrzesiadkami, Zniżki.

4. Zarządzanie usługami dodatkowymi

Funkcjonalność dedykowana dla biletów międzykontynentalnych, pozwalająca na personalizację podróży.

Dodawanie posiłków: Rejestrowanie wybranych potraw przez pasażera.

Raportowanie usług: Wyświetlanie czytelnej listy wybranych udogodnień w osobnych liniach (pętla foreach).

- **Klasy:** IUslugowy, BiletMiedzykontynentalny.

5. Zarządzanie danymi

Zapewnienie trwałości rezerwacji oraz integracja z zewnętrznymi źródłami danych.

Serializacja XML: Zapis i odczyt całej listy biletów do pliku tekstowego, co pozwala na przenoszenie danych między systemami.

Integracja z Bazą Danych (ORM): Trwały zapis obiektów w relacyjnej bazie danych za pomocą Entity Framework.

Klonowanie rezerwacji: Szybkie tworzenie biletów dla grup (np. rodziny) przy zachowaniu tych samych parametrów lotu, ale nowej tożsamości pasażera.

- **Klasy:** Bilet, BiletDbContext, XmlSerializer.

6. System walidacji i bezpieczeństwa

Zapewnienie poprawności wprowadzanych danych i ochrona przed błędami logicznymi.

Walidacja czasu: Blokowanie rezerwacji na loty z datą wsteczną.

Walidacja danych geograficznych: Wymuszanie poprawnej długości nazw miast.

- **Klasy:** BlednaDataLotuException, BledneMiastoException.

7. Inteligentny interfejs użytkownika (GUI)

Dynamiczne Formularze: Pola takie jak "Wymagana wiza" czy "Wybór posiłków" pojawiają się tylko dla odpowiednich typów biletów (SelectionChanged).

Podgląd Live: Użytkownik widzi aktualną cenę biletu przed dokonaniem zakupu, co eliminuje pomyłki.

Zarządzanie Listą: Przeglądanie wszystkich zakupionych biletów w widoku ListView oraz sprawdzanie ich szczegółów.

- **Klasy:** MainWindow, ProjektGUI.

8. Kontrola jakości (testy)

Integralną częścią projektu jest zestaw testów jednostkowych, które weryfikują poprawność działania bez konieczności ręcznego klikania w GUI.

Testy Logiki: Sprawdzanie, czy zniżki i podatki sumują się poprawnie.

Testy Wyjątków: Weryfikacja czy system poprawnie reaguje na błędne dane.

- **Klasy:** BiletKrajowyTests, BiletMiedzykontynentalnyTests, BrakMiejsceExceptionTests.